

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Paralelní Programování v Go



2026

Vedoucí práce:
Mgr. Jan Laštovička, Ph.D.

Richard Bebjak

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor:	Richard Bebjak
Název práce:	Paralelní Programování v Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Jan Laštovička, Ph.D.
Počet stran:	10
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Richard Bebjak
Title:	Parallel programming in Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Jan Laštovička, Ph.D.
Page count:	10
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Synopsis

Kľúčové slová:

Keywords:

Děkuji, děkuji, děkuji.

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Komunikačné kanály	7
1.1	Bufferované a nebufferované komunikačné kanály	8
1.2	Jednosmerný komunikačný kanál	8
2	Synchronizačné primitíva	8
2.1	WaitGroup	9
2.2	Select	9
2.3	Go pamäťový model	9
3	Chat server	9
3.1	Funkčné požiadavky	9
3.2	Súbežná architektúra	9
4	Paralelné algoritmy	9
5	Paralelné programovanie	9
5.1	Paralelné a súbežné programovanie	9
5.2	Problémy súbežného programovania	9
6	Jazyk Go	9
6.1	Goroutines	9
	Literatúra	10

Zoznam tabuliek

1 Komunikačné kanály

Komunikačné kanály predstavujú jeden zo základných mechanizmov synchronizácie a komunikácie medzi goroutines. Sú zabudované referečným typom, ktorý umožňuje bezpečný prenos dát. Ich návrh vychádza z Communicating Sequential Processes, ktorý zdôrazňuje komunikáciu medzi procesmi miesto zdieľania pamäte.^[1]

Komunikačné kanály sa deklarujú pomocou kľúčového slova `chan` a vždy sa deklarujú aj s určitým typom. To znamená, že kanál môže prenášať hodnoty iba tohoto typu. Komunikačné kanály patria k referečným typom, preto ich deklarácia nevytvára potrebnú pamäťovú štruktúru. Pamäť sa alokuje pomocou zabudovanej funkcie `make`.

```
make(chan type , buffer_size)
```

Takto vytvorený komunikačný kanál umožňuje zapisovanie a čítanie hodnôt. Zápis do komunikačného kanála umožňuje operátor `<-`, ktorý sa umiestňuje na pravú stranu názvu komunikačného kanála, pre čítanie sa operátor umiestňuje na ľavú stranu názvu komunikačného kanála.

```
channel <- data
variable = <- channel
```

Dôležitou vlastnosťou komunikačného kanála je jeho blokujúce správanie. Ak goroutine zapisuje do komunikačného kanála, ktorý je plný, je jej proces blokovaný kým iná goroutine z komunikačného kanála hodnotu neprečíta. Podobne čítanie z prázdneho komunikačného kanála je blokovaný kým nie je do komunikačného kanála zapísaná nová hodnota.

Komunikačný kanál je možné zavrieť pomocou funkcie `close`. Pri čítaní z kanála je možné získať druhú návratovú hodnotu typu `bool`, ktorá indikuje jeho otvorenie. Uzavretie teda sygnalizuje príjmateľovi, že do komunikačného kanála nebude viac zapisované. Ak odosielateľ pošle data do uzavretého komunikačného kanálu, príjmateľ obdrží `error send on closed channel`.

```
var channel = make(chan int , 1)

go func(){
    channel <- 42
}()

result := <- channel
close(channel)
```

Na riadku jedna si vytvoríme komunikačný kanál s referečným typom `int` a veľkosťou buffera jedna. Na riadku tri spustíme novú goroutine. Na riadku štyri v goroutine uložíme do komunikačného kanála hodnotu 42. Na riadku 7 hlavné vlákno čaká na data z komunikačného kanála. Ak dáta prídu, uloží do premennej "result" data z komunikačného kanála, v tomto prípade 42. Na riadku 8 uzavrieme komunikačný kanál.

1.1 Bufferované a nebufferované komunikačné kanály

Každý komunikačný kanál môže byť bufferovaný alebo nebufferovaný. Nebufferovaný komunikačný kanál nemá žiadnu kapacitu a každá operácia zápisu musí byť okamžite spravovaná operáciou čítania.

Bufferovaný komunikačný kanál sa definuje pomocou voliteľného druhého argumentu funkcie `make`. Bufferovaný komunikačný kanál obsahuje bugger s definovanou kapacitou, ktorá umožní uložiť určitý počet hodnôt do buffera bez okamžitého čítania. Bufferovaný komunikačný kanál zablokuje zápis až v momente keď je buffer plný.

1.2 Jednosmerný komunikačný kanál

Jazyk Go umožňuje definovanie jednosmerných kanálov, ktoré podporujú iba zápis alebo čítanie. Tieto komunikačné kanály sa používajú ako parametre funkcií, aby určili akým spôsobom sa komunikačný kanál v danej časti programu používa. Tento mechanizmus zvyšuje bezpečnosť programu, pretože kompilátor dokáže zachytiť nesprávne použitie.

2 Synchronizačné primitíva

Jazyk Go poskytuje `sync package`, ktorý obsahuje synchronizačné primitíva. Tieto synchronizačné primitíva sú používané pri nízko úrovňovom prístupe do pamäti. Rozdiel medzi ostatnými jazykmi a jazykom Go je, že Go poskytuje novú sadu synchronizačných primitív pre súbežnosť nad sádov primitív pre synchronizáciu prístupu k pamäti.

- 2.1 WaitGroup
- 2.2 Select
- 2.3 Go pamäťový model
- 3 Chat server
 - 3.1 Funkčné požiadavky
 - 3.2 Súbežná architektúra
- 4 Paralelné algoritmy
- 5 Paralelné programovanie
 - 5.1 Paralelné a súbežné programovanie
 - 5.2 Problémy súbežného programovania
- 6 Jazyk Go
 - 6.1 Goroutines

Literatúra

- [1] Cox-Buday, Katherine. *Concurrency in Go*. First. Sebastopol, CA: O'Reilly Media, 2017. 224 str. ISBN 978-1-491-94119-5.